



Database systems

ITCS2322

Chapter 10 : Transaction Management and Concurrency Control

اعداد : دكتور محمد عبد اللطيف راضي

Learning Objectives

- In this chapter, students will learn:
 - About database transactions and their properties
 - What concurrency control is and what role it plays in maintaining the database's integrity
 - What locking methods are and how they work

Learning Objectives

- In this chapter, students will learn:
 - How stamping methods are used for concurrency control
 - How optimistic methods are used for concurrency control
 - How database recovery management is used to maintain database integrity

Chapter 5: Advanced Data Modeling

Table of contents

- What Is a Transaction?
- Concurrency Control Concurrency Control with Locking
- Concurrency Control with Time Stamping Methods
- Concurrency Control with Optimistic Methods
- ANSI Levels of Transaction Isolation
- Database Recovery Management

Transaction

- Logical unit of work that must be entirely completed or aborted
- Consists of:
 - SELECT statement
 - Series of related UPDATE statements
 - Series of INSERT statements
 - Combination of SELECT, UPDATE, and INSERT statements

Transaction

- Consistent database state: All data integrity constraints are satisfied
 - Must begin with the database in a known consistent state to ensure consistency
- Formed by two or more database requests
 - Database requests: Equivalent of a single SQL statement in an application program or transaction
- Consists of a single SQL statement or a collection of related SQL statements

Evaluating Transaction Results

Transaction can be simple SQL code:

```
SELECT      CUST_NUMBER, CUST_BALANCE
FROM        CUSTOMER
WHERE       CUST_NUMBER = 10016;
```

Transaction may consist of a single SQL statement or a collection of related SQL statements

```
INSERT INTO INVOICE
VALUES (1009, 10016, '18-Jan-2018', 256.99, 20.56, 277.55, 'cred', 0.00, 277.55);
INSERT INTO LINE
VALUES (1009, 1, '89-WRE-Q', 1, 256.99, 256.99);

UPDATE PRODUCT
SET   PROD_QOH = PROD_QOH - 1
WHERE PROD_CODE = '89-WRE-Q';

UPDATE CUSTOMER
SET   CUST_BALANCE = CUST_BALANCE + 277.55
WHERE CUST_NUMBER = 10016;

INSERT INTO ACCT_TRANSACTION
VALUES (10007, '18-Jan-18', 10016, 'charge', 277.55);

COMMIT;
```


Evaluating Transaction Results

FIGURE 10.2 TRACING THE TRANSACTION IN THE CH10_SALECO DATABASE

Table name: INVOICE

INV NUMBER	CUST NUMBER	INV DATE	INV SUBTOTAL	INV TAX	INV TOTAL	INV PAY TYPE	INV PAY AMOUNT	INV BALANCE
1001	10014	18-Jan-18	54.52	0.59	55.11	cc	55.11	0.00
1002	10011	18-Jan-18	9.90	0.90	10.79	cash	10.79	0.00
1003	10012	18-Jan-18	270.73	27.08	297.81	cc	297.81	0.00
1004	10011	17-Jan-18	34.87	2.59	37.46	cc	37.46	0.00
1005	10018	17-Jan-18	70.44	6.84	77.28	cc	77.28	0.00
1006	10014	17-Jan-18	307.03	31.03	338.06	credit	338.06	338.06
1007	10015	17-Jan-18	34.97	2.80	37.77	cash	37.77	0.00
1008	10011	17-Jan-18	1033.00	92.65	1125.65	credit	1125.65	1125.65
1009	10018	18-Jan-18	298.00	30.90	328.90	cash	0.00	328.90

Table name: PRODUCT

PROD CODE	PROD DESCRIPTION	PROD UOM	PROD QTY	PROD AMT	PROD PRICE	PROD DISCOUNT	VEND NUMBER
1168P01	Printer paper, 1500, 3-month		8	5	625.00	0.00	26501
13-03P01	2.5in. glue stick 1000		32	15	14.06	0.00	26344
14-01A1	SCN41 glue stick 1000		18	12	17.49	0.00	26344
1545-002	ink color, 14-in, 2-in		15	8	30.95	0.00	22119
1556-001	ink color, 14-in, 2-in		20	5	40.99	0.00	22119
2230P01	860-gram, 12-in. wide		6	5	839.93	0.00	24399
2230P02	860-gram, 8-in. wide		6	5	99.97	0.00	24399
2230P03	860-gram, 14-in. wide		12	5	39.95	0.00	22119
23-09-001	Clear tape		25	10	9.95	0.10	26325
23-14-001	Sticker paper, 12 in.		8	5	14.40	0.00	
54775-01	Ballpoint pen, 1/8 in.		40	20	4.99	0.00	25344
54775-02	Ballpoint pen, 1/8 in.		10	5	299.99	0.00	26325
PVC33041	PVC pipe, 3.5 in., 8 ft.		180	75	5.87	0.00	
3M-10277	1.25-in. metal screw, 25		172	75	5.99	0.00	26325
3M-23416	2.5-in. metal screw, 50		237	180	8.45	0.00	26321
4853P03	Scallop, 4-in. x 1/2 in., 20 each		18	8	615.00	0.10	58031

Table name: CUSTOMER

CUST NUM	CUST NAME	CUST PHONE	CUST EMAIL	CUST ADDRESS	CUST PHONE	CUST BALANCE
10010	Parsons	Alfred	A	515	344-2573	0.00
10011	Donne	Leona	K	713	894-1238	615.73
10012	Smith	Paul	W	515	894-2295	0.00
10013	Olson	Paul	C	515	894-2560	0.00
10014	Orlando	Myron		515	222-9572	0.00
10015	O'Brien	Arny	B	713	442-3361	0.00
10016	Brown	James	G	515	297-1238	277.95
10017	Williams	George		515	290-2566	0.00
10018	Farris	Anna	G	713	352-7105	0.00
10019	Smith	Chelle	K	515	297-3339	0.00

Table name: ACCT_TRANSACTION

ACCT TRANS NUM	ACCT TRANS DATE	CUST NUMBER	ACCT TRANS TYPE	ACCT TRANS AMOUNT
10001	17-Jan-18	10014	charge	329.66
10004	17-Jan-18	10011	charge	415.73
10006	26-Jan-18	10014	payment	329.66
10007	18-Jan-18	10016	charge	277.95

Database name: Ch10_SaleCo

Table name: LINE

INV NUMBER	LINE NUMBER	PROD CODE	LINE UNITS	LINE PRICE	LINE AMOUNT
1001	1	13-03P01	3	14.06	42.18
1001	2	14-01A1	1	3.96	3.96
1002	1	154775-01	2	4.99	9.98
1003	1	13230P01	8	30.95	247.60
1003	2	1545-002	1	39.95	39.95
1003	3	15-03P01	5	14.96	74.80
1004	1	154775-01	3	4.99	14.97
1004	2	13230P01	2	30.95	61.90
1005	1	PVC33041	12	5.87	70.44
1006	1	3M-10277	3	5.99	17.97
1006	2	2230P01	1	100.92	100.92
1006	3	21-03P01	1	3.96	3.96
1006	4	4853P01	1	269.99	269.99
1007	1	13-03P01	2	14.06	28.12
1007	2	154775-01	1	4.99	4.99
1008	1	PVC33041	5	5.87	29.35
1008	2	1545-002	6	119.96	719.56
1008	3	21-03P01	1	3.96	3.96
1008	4	4853P01	2	269.99	539.98
1009	1	13230P01	1	269.99	269.99

Evaluating Transaction Results

- Not all transactions update database
 - SQL code represents a transaction because it accesses a database
- Improper or incomplete transactions can have devastating effect on database integrity
 - Users can define enforceable constraints based on business rules
 - Other integrity rules are automatically enforced by the DBMS

Transaction Properties

Atomicity

- All operations of a transaction must be completed
- If not, the transaction is aborted

Consistency

- Permanence of database's consistent state

Isolation

- Data used during transaction cannot be used by second transaction until the first is completed

Durability

- Ensures that once transactions are committed, they cannot be undone or lost

Serializability

- Ensures that the schedule for the concurrent execution of several transactions should yield consistent results

Transaction Management with SQL

- SQL statements that provide transaction support
 - COMMIT
 - ROLLBACK
- Transaction sequence must continue until:
 - COMMIT statement is reached
 - ROLLBACK statement is reached
 - End of program is reached
 - Program is abnormally terminated

Transaction Log

- Keeps track of all transactions that update the database
- DBMS uses the information stored in a log for:
 - Recovery requirement triggered by a ROLLBACK statement
 - A program's abnormal termination
 - A system failure

Transaction Log

- The transaction log stores the following:
- A record for the beginning of the transaction.
- For each transaction component (SQL statement):
 - The type of operation being performed (INSERT, UPDATE, DELETE).
 - The names of the objects affected by the transaction (the name of the table).
 - The “before” and “after” values for the fields being updated.
 - Pointers to the previous and next transaction log entries for the same transaction.
- The ending (COMMIT) of the transaction.

Transaction Log

TABLE 10.1

A TRANSACTION LOG

TRL_ID	TRX_NUM	PREV_PTR	NEXT_PTR	OPERATION	TABLE	ROW ID	ATTRIBUTE	BEFORE VALUE	AFTER VALUE
341	101	Null	352	START	****Start Transaction				
352	101	341	363	UPDATE	PRODUCT	1558-QW1	PROD_QOH	25	23
363	101	352	365	UPDATE	CUSTOMER	10011	CUST_BALANCE	525.75	615.73
365	101	363	Null	COMMIT	**** End of Transaction				



TRL_ID = Transaction log record ID

TRX_NUM = Transaction number

PTR = Pointer to a transaction log record ID

(Note: The transaction number is automatically assigned by the DBMS.)

Concurrency Control

- Coordination of the simultaneous transactions execution in a multiuser database system
- Objective - Ensures serializability of transactions in a multiuser database environment

Problems in Concurrency Control

Lost update

- Occurs in two concurrent transactions when:
 - Same data element is updated
 - One of the updates is lost

Uncommitted data

- Occurs when:
 - Two transactions are executed concurrently
 - First transaction is rolled back after the second transaction has already accessed uncommitted data

Inconsistent retrievals

- Occurs when a transaction accesses data before and after one or more other transactions finish working with such data

Lost update

TABLE 10.2

TWO CONCURRENT TRANSACTIONS TO UPDATE QOH

TRANSACTION	COMPUTATION
T1: Purchase 100 units	$PROD_QOH = PROD_QOH + 100$
T2: Sell 30 units	$PROD_QOH = PROD_QOH - 30$

Table 10.3 shows the serial execution of the transactions under normal circumstances, yielding the correct answer $PROD_QOH = 105$.

TABLE 10.3

SERIAL EXECUTION OF TWO TRANSACTIONS

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read $PROD_QOH$	35
2	T1	$PROD_QOH = 35 + 100$	
3	T1	Write $PROD_QOH$	135
4	T2	Read $PROD_QOH$	135
5	T2	$PROD_QOH = 135 - 30$	
6	T2	Write $PROD_QOH$	105

Lost update

- However, suppose that a transaction can read a product's PROD_QOH value from the table before a previous transaction has been committed, using the same product.
- The sequence depicted in Table 10.4 shows how the lost update problem can arise.
- Note that the first transaction (T1) has not yet been committed when the second transaction (T2) is executed.
- Therefore, T2 still operates on the value 35, and its subtraction yields 5 in memory.
- In the meantime, T1 writes the value 135 to disk, which is promptly overwritten by T2. In short, the addition of 100 units is “lost” during the process.

Lost update

TABLE 10.4

LOST UPDATES

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T2	Read PROD_QOH	35
3	T1	$PROD_QOH = 35 + 100$	
4	T2	$PROD_QOH = 35 - 30$	
5	T1	Write PROD_QOH (lost update)	135
6	T2	Write PROD_QOH	5

Uncommitted data

TABLE 10.5

TRANSACTIONS CREATING AN UNCOMMITTED DATA PROBLEM

TRANSACTION	COMPUTATION
T1: Purchase 100 units	PROD_QOH = PROD_QOH + 100 (Rolled back)
T2: Sell 30 units	PROD_QOH = PROD_QOH - 30

Table 10.6 shows how the serial execution of these transactions yields the correct answer under normal circumstances.

TABLE 10.6

CORRECT EXECUTION OF TWO TRANSACTIONS

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T1	PROD_QOH = 35 + 100	
3	T1	Write PROD_QOH	135
4	T1	*****ROLLBACK *****	35
5	T2	Read PROD_QOH	35
6	T2	PROD_QOH = 35 - 30	
7	T2	Write PROD_QOH	5

Uncommitted data

Table 10.7 shows how the uncommitted data problem can arise when the ROLLBACK is completed after T2 has begun its execution.

TABLE 10.7

AN UNCOMMITTED DATA PROBLEM

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T1	$PROD_QOH = 35 + 100$	
3	T1	Write PROD_QOH	135
4	T2	Read PROD_QOH (Read uncommitted data)	135
5	T2	$PROD_QOH = 135 - 30$	
6	T1	***** ROLLBACK *****	35
7	T2	Write PROD_QOH	105

Inconsistent retrievals

TABLE 10.8

RETRIEVAL DURING UPDATE

TRANSACTION T1	TRANSACTION T2
SELECT SUM(PROD_QOH) FROM PRODUCT	UPDATE PRODUCT SET PROD_QOH = PROD_QOH + 10 WHERE PROD_CODE = 1546-QQ2
	UPDATE PRODUCT SET PROD_QOH = PROD_QOH - 10 WHERE PROD_CODE = 1558-QW1
	COMMIT;

- While T1 calculates the total quantity on hand (PROD_QOH) for all items, T2 represents the correction of a typing error: the user added 10 units to product 1558-QW1's PROD_QOH but meant to add the 10 units to product 1546-QQ2's PROD_QOH.
- To correct the problem, the user adds 10 to product 1546-QQ2's PROD_QOH and subtracts 10 from product 1558-QW1's PROD_QOH.
- The initial and final PROD_QOH values are reflected in Table 10.9. (Only a few PROD_CODE values are shown for the PRODUCT table. To illustrate the point, the sum for the PROD_QOH values is shown for these few products.)

Inconsistent retrievals

TABLE 10.9

TRANSACTION RESULTS: DATA ENTRY CORRECTION

	BEFORE	AFTER
PROD_CODE	PROD_QOH	PROD_QOH
11QER/31	8	8
13-Q2/P2	32	32
1546-QQ2	15	$(15 + 10) \rightarrow 25$
1558-QW1	23	$(23 - 10) \rightarrow 13$
2232-QTY	8	8
2232-QWE	6	6
Total	92	92

Inconsistent retrievals

TABLE 10.10

INCONSISTENT RETRIEVALS

TIME	TRANSACTION	ACTION	VALUE	TOTAL
1	T1	Read PROD_QOH for PROD_CODE = '11QER/31'	8	8
2	T1	Read PROD_QOH for PROD_CODE = '13-Q2/P2'	32	40
3	T2	Read PROD_QOH for PROD_CODE = '1546-QQ2'	15	
4	T2	PROD_QOH = 15 + 10		
5	T2	Write PROD_QOH for PROD_CODE = '1546-QQ2'	25	
6	T1	Read PROD_QOH for PROD_CODE = '1546-QQ2'	25	(After) 65
7	T1	Read PROD_QOH for PROD_CODE = '1558-QW1'	23	(Before) 88
8	T2	Read PROD_QOH for PROD_CODE = '1558-QW1'	23	
9	T2	PROD_QOH = 23 - 10		
10	T2	Write PROD_QOH for PROD_CODE = '1558-QW1'	13	
11	T2	***** COMMIT *****		
12	T1	Read PROD_QOH for PROD_CODE = '2232-QTY'	8	96
13	T1	Read PROD_QOH for PROD_CODE = '2232-QWE'	6	102

The Scheduler

- Establishes the order in which the operations are executed within concurrent transactions
- Interleaves the execution of database operations to ensure serializability and isolation of transactions
- Based on concurrent control algorithms to determine the appropriate order
- Creates serialization schedule
 - Serializable schedule: Interleaved execution of transactions yields the same results as the serial execution of the transactions

The Scheduler

TABLE 10.11

READ/WRITE CONFLICT SCENARIOS: CONFLICTING DATABASE OPERATIONS MATRIX

	TRANSACTIONS		RESULT
	T1	T2	
Operations	Read	Read	No conflict
	Read	Write	Conflict
	Write	Read	Conflict
	Write	Write	Conflict

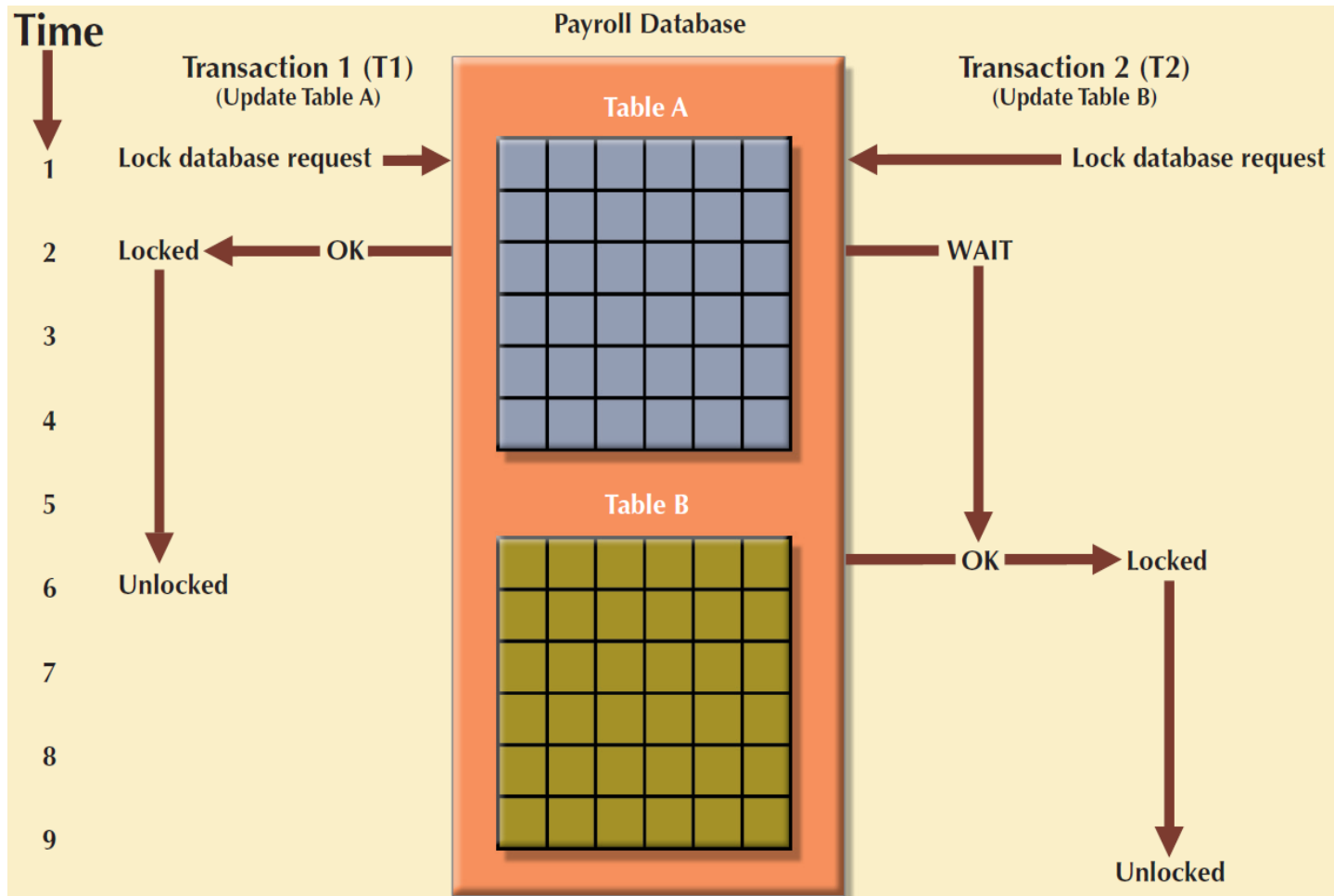
Concurrency Control with Locking Methods

- Locking methods - Facilitate isolation of data items used in concurrently executing transactions
- Lock: Guarantees exclusive use of a data item to a current transaction
- Pessimistic locking: Use of locks based on the assumption that conflict between transactions is likely
- Lock manager: Responsible for assigning and policing the locks used by the transactions

Lock Granularity

- Indicates the level of lock use
- Levels of locking
 - Database-level lock
 - Table-level lock
 - Page-level lock
 - Page or diskpage: Directly addressable section of a disk
 - Row-level lock
 - Field-level lock

Figure 10.3 - Database-Level Locking Sequence



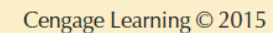
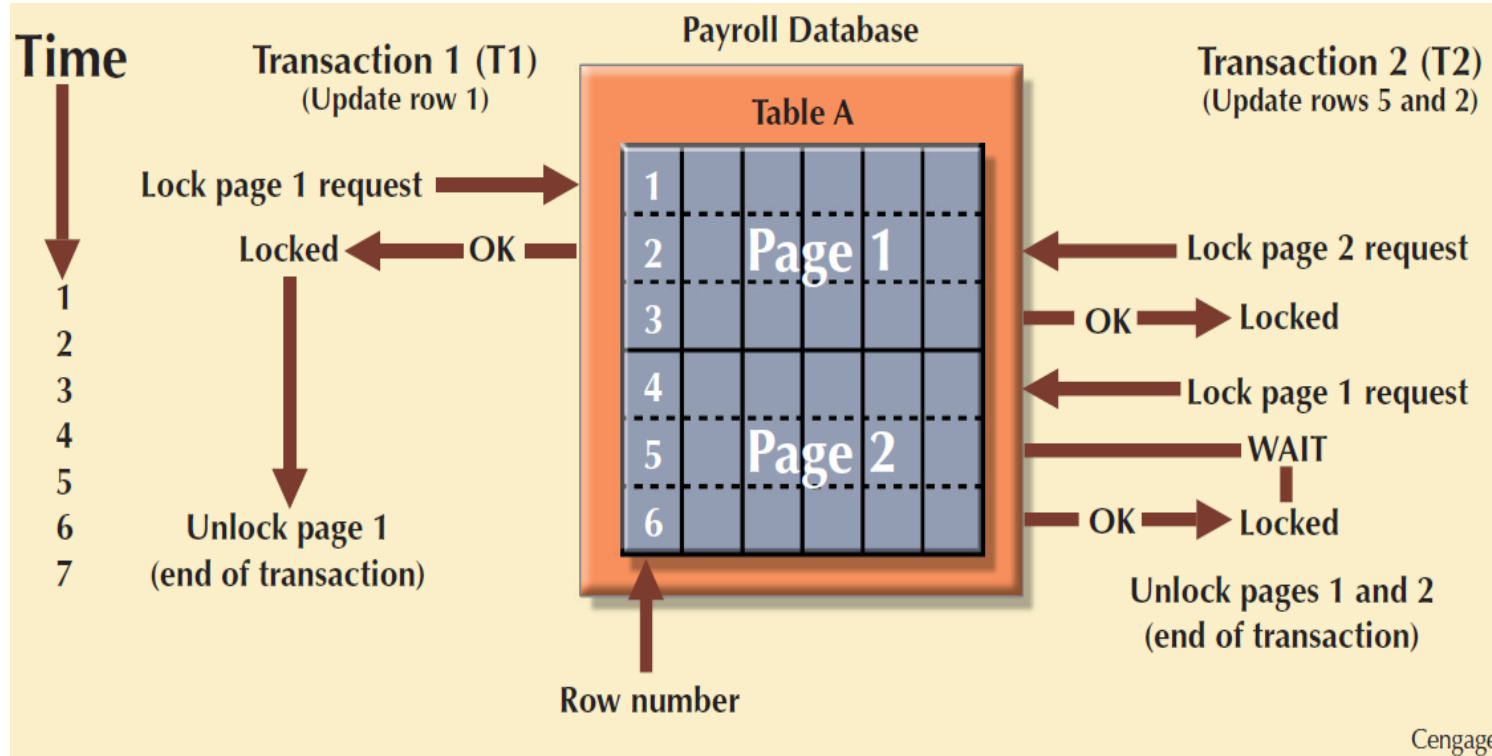
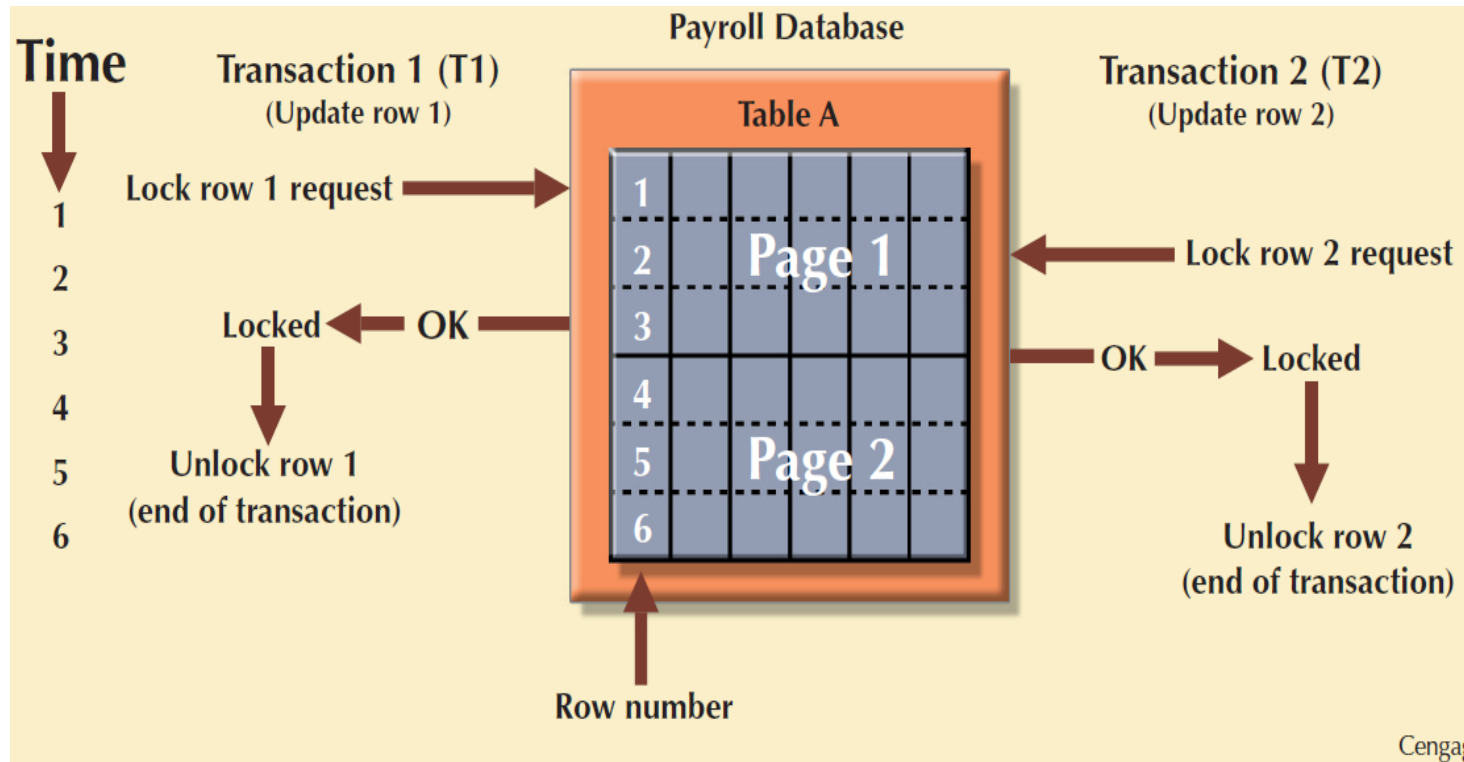


Figure 10.5 - An Example of a Page-Level Lock



Cengage Learning © 2015

Figure 10.6 - An Example of a Row-Level Lock



Cengage Learning © 2015

Lock Types

Binary lock

- Has two states, locked (1) and unlocked (0)
- If an object is locked by a transaction, no other transaction can use that object
- If an object is unlocked, any transaction can lock the object for its use

Exclusive lock

- Exists when access is reserved for the transaction that locked the object

Shared lock

- Exists when concurrent transactions are granted read access on the basis of a common lock

Problems in Using Locks

- Resulting transaction schedule might not be serializable
- Schedule might create deadlocks

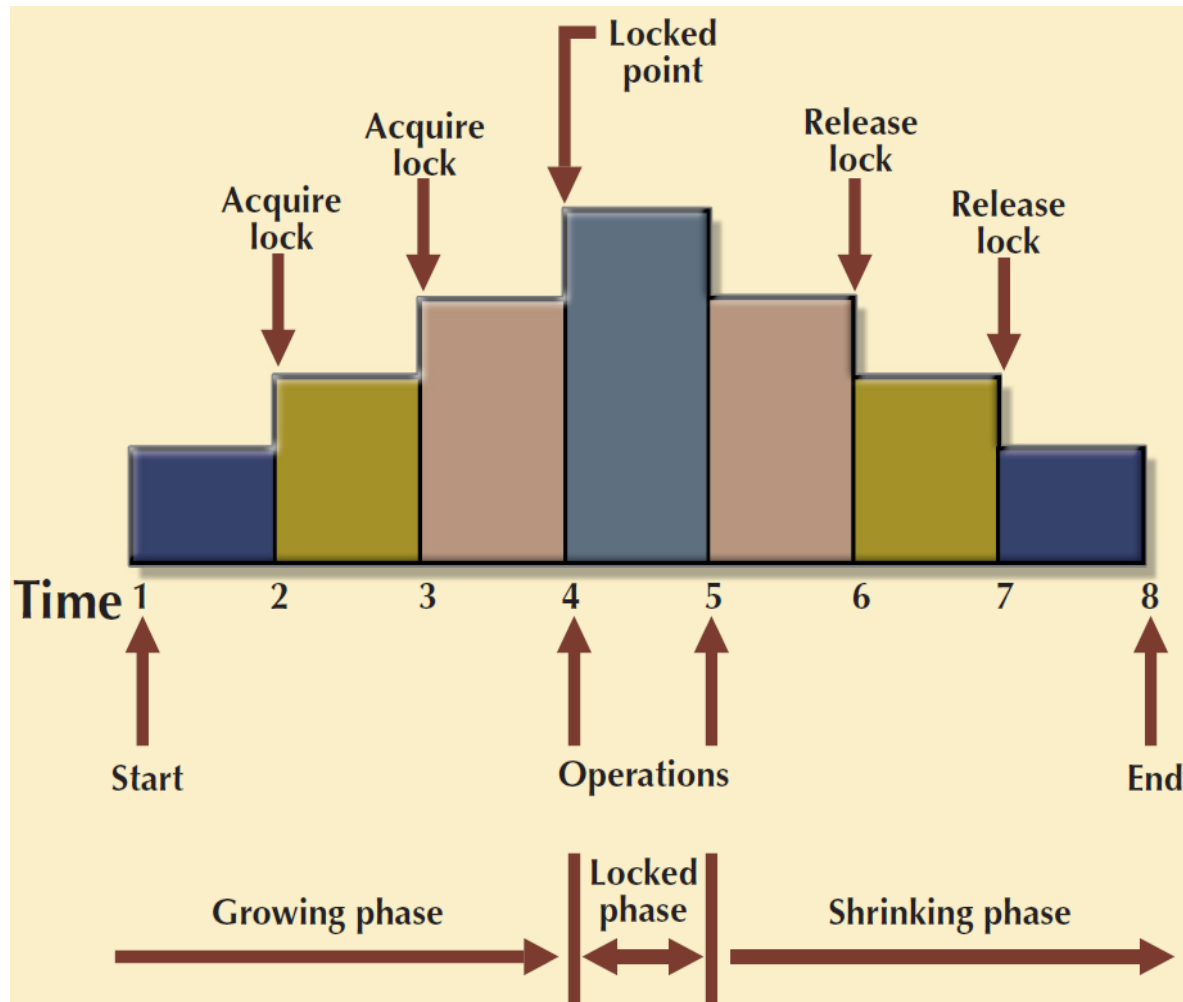
Two-Phase Locking (2PL)

- Defines how transactions acquire and relinquish locks
- Guarantees serializability but does not prevent deadlocks
- Phases
 - Growing phase - Transaction acquires all required locks without unlocking any data
 - Shrinking phase - Transaction releases all locks and cannot obtain any new lock

Two-Phase Locking (2PL)

- Governing rules
 - Two transactions cannot have conflicting locks
 - No unlock operation can precede a lock operation in the same transaction
 - No data are affected until all locks are obtained

Figure 10.7 - Two-Phase Locking Protocol

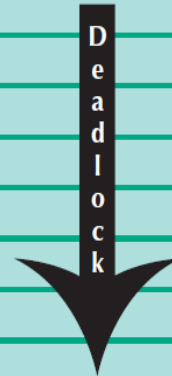


Deadlocks

- Occurs when two transactions wait indefinitely for each other to unlock data
 - Known as deadly embrace
- Control techniques
 - Deadlock prevention
 - Deadlock detection
 - Deadlock avoidance
- Choice of deadlock control method depends on database environment

Table 10.13 - How a Deadlock Condition is Created

TIME	TRANSACTION	REPLY	LOCK STATUS	
0			Data X	Data Y
1	T1:LOCK(X)	OK	Unlocked	Unlocked
2	T2: LOCK(Y)	OK	Locked	Unlocked
3	T1:LOCK(Y)	WAIT	Locked	Locked
4	T2:LOCK(X)	WAIT	Locked	Locked
5	T1:LOCK(Y)	WAIT	Locked	Locked
6	T2:LOCK(X)	WAIT	Locked	Locked
7	T1:LOCK(Y)	WAIT	Locked	Locked
8	T2:LOCK(X)	WAIT	Locked	Locked
9	T1:LOCK(Y)	WAIT	Locked	Locked
...				
...				
...				
...				



Cengage Learning © 2015

Time Stamping

- Assigns global, unique time stamp to each transaction
 - Produces explicit order in which transactions are submitted to DBMS
- Properties
 - Uniqueness: Ensures no equal time stamp values exist
 - Monotonicity: Ensures time stamp values always increases

Time Stamping

- Disadvantages
 - Each value stored in the database requires two additional stamp fields
 - Increases memory needs
 - Increases the database's processing overhead
 - Demands a lot of system resources

Table 10.14 - Wait/Die and Wound/Wait Concurrency Control Schemes

TRANSACTION REQUESTING LOCK	TRANSACTION OWNING LOCK	WAIT/DIE SCHEME	WOUND/WAIT SCHEME
T1 (11548789)	T2 (19562545)	T1 waits until T2 is completed and T2 releases its locks.	- T1 preempts (rolls back) T2. - T2 is rescheduled using the same timestamp.
T2 (19562545)	T1 (11548789)	- T2 dies (rolls back). - T2 is rescheduled using the same times tamp.	T2 waits until T1 is completed and T1 releases its locks.

Cengage Learning © 2015

Concurrency Control with Optimistic Methods

- Optimistic approach: Based on the assumption that the majority of database operations do not conflict
 - Does not require locking or time stamping techniques
 - Transaction is executed without restrictions until it is committed

Phases of Optimistic Approach

Read

- Transaction:
 - Reads the database
 - Executes the needed computations
 - Makes the updates to a private copy of the database values

Validation

- Transaction is validated to ensure that the changes made will not affect the integrity and consistency of the database

Write

- Changes are permanently applied to the database

Table 10.15 - Transaction Isolation Levels

	Isolation Level	Allowed			Comment
		Dirty Read	Nonrepeatable Read	Phantom Read	
Less restrictive ↑↓ More restrictive	Read Uncommitted	Y	Y	Y	The transaction reads uncommitted data, allows nonrepeatable reads and, phantom reads.
	Read Committed	N	Y	Y	Does not allow uncommitted data reads but allows nonrepeatable reads and phantom reads.
	Repeatable Read	N	N	Y	Only allows phantom reads.
	Serializable	N	N	N	Does not allow dirty reads, nonrepeatable reads, or phantom reads.
Oracle / SQL Server Only	Read Only / Snapshot	N	N	N	Supported by Oracle and SQL Server. The transaction can only see the changes that were committed at the time the transaction started.

Cengage Learning © 2015

Database Recovery Management

- Database recovery: Restores database from a given state to a previously consistent state
- Recovery transactions are based on the atomic transaction property
- Atomic transaction property: All portions of a transaction must be treated as a single logical unit of work
 - If transaction operation cannot be completed:
 - Transaction must be aborted
 - Changes to database must be rolled back

Concepts that Affect Transaction Recovery

Deferred-write technique or deferred update

- Ensures that transaction logs are always written before the data are updated

Redundant transaction logs

- Ensure that a physical disk failure will not impair the DBMS's ability to recover data

Buffers

- Temporary storage areas in a primary memory

Checkpoints

- Allows DBMS to write all its updated buffers in memory to disk

Techniques used in Transaction Recovery Procedures

Deferred-write technique or deferred update

- Only transaction log is updated

Write-through technique or immediate update

- Database is immediately updated by transaction operations during transaction's execution

Recovery Process in Deferred-Write Technique

- Identify the last check point in the transaction log
- If transaction was committed before the last check point
 - Nothing needs to be done
- If transaction was committed after the last check point
 - Transaction log is used to redo the transaction
- If transaction had a ROLLBACK operation after the last check point
 - Nothing needs to be done

Recovery Process in Write-Through Technique

- Identify the last checkpoint in the transaction log
- If transaction was committed before the last checkpoint
 - Nothing needs to be done
- If transaction was committed after the last checkpoint
 - Transaction must be redone
- If transaction had a ROLLBACK operation after the last check point
 - Transaction log is used to ROLLBACK the operations